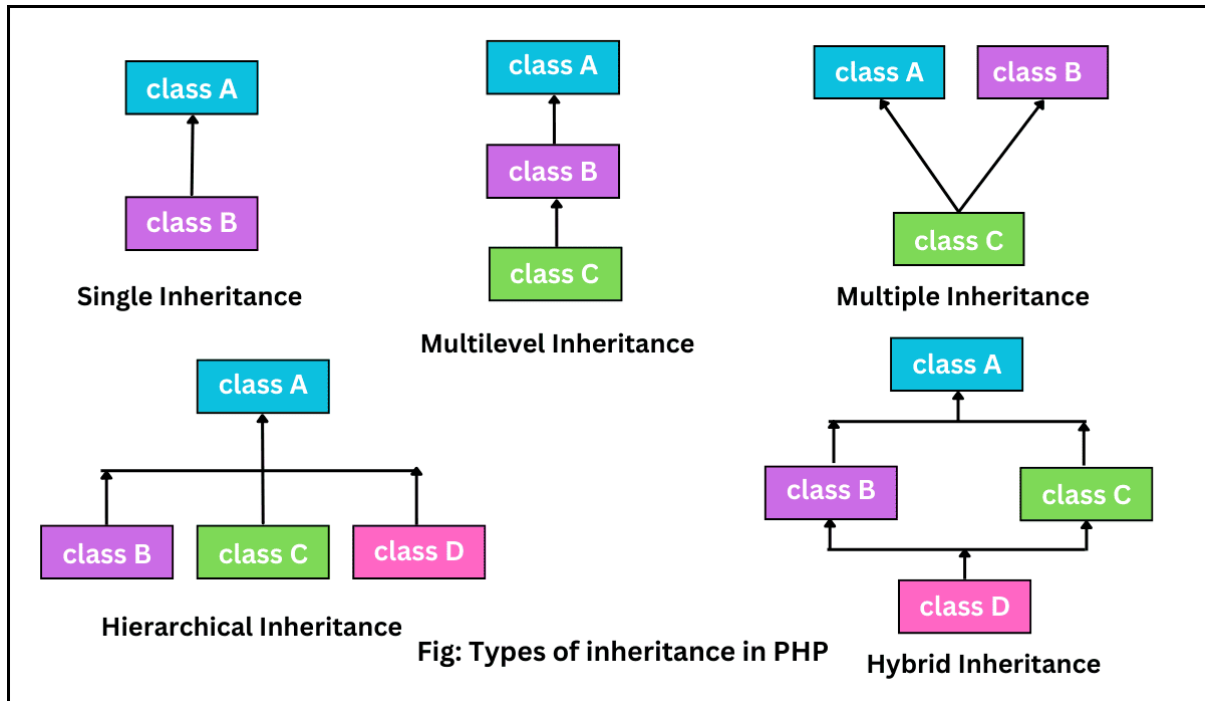


Inheritance

Inheritance is a mechanism in Java that allows one class (child/subclass) to inherit the properties and methods of another class (parent/superclass).

Types of Inheritance



Single Inheritance

One child class inherits from one parent class

```
class Animal {  
    void eat() {  
        System.out.println("Animal eats");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.eat();  
        d.bark();  
    }  
}
```

Animal eats
Dog barks

Multilevel Inheritance

A class inherits from another class, which is also inherited by another class.

```
class Animal {
    void eat() {
        System.out.println("Animal eats");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}

class Puppy extends Dog {
    void weep() {
        System.out.println("Puppy weeps");
    }
}

public class Main {
    public static void main(String[] args) {
        Puppy p = new Puppy();
        p.eat();
        p.bark();
        p.weep();
    }
}
```

Animal eats
Dog barks
Puppy weeps

Hierarchical Inheritance

Multiple child classes inherit from the same parent class

```
class Animal {
    void eat() {
        System.out.println("Animal eats");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal {
    void meow() {
        System.out.println("Cat meows");
    }
}
```

```

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        Cat c = new Cat();

        d.eat();
        d.bark();

        c.eat();
        c.meow();
    }
}

```

Animal eats
 Dog barks
 Animal eats
 Cat meows

Multiple Inheritance (Using Interfaces)

Java does not support multiple inheritances with classes, but it supports it through interfaces.

```

interface A {
    void displayA();          // public static void displayA();
}

interface B {
    void displayB();          // public static void displayB();
}

class C implements A, B {
    public void displayA() {
        System.out.println("Interface A");
    }

    public void displayB() {
        System.out.println("Interface B");
    }
}

public class Main {
    public static void main(String[] args) {
        C obj = new C();
        obj.displayA();
        obj.displayB();
    }
}

```

Interface A
 Interface B

Visibility in Method Overriding

Parent Method	Child Method	Allowed?
public	public	✓ Yes
public	protected	☐ No
public	default	☐ No
public	private	☐ No
protected	protected	✓ Yes
protected	public	✓ Yes
protected	default	☐ No
default	default	✓ Yes
default	public	✓ Yes
default	protected	✓ Yes
default	private	☐ No

Hybrid Inheritance

Hybrid inheritance is a combination of two or more inheritance types. It is **not supported with classes** in Java because multiple inheritances of classes are not allowed. However, it can be achieved using **interfaces**.

```
interface A {
    void displayA();
}

interface B extends A {
}

interface C {
    void displayC();
}

class D implements B, C { // Multiple inheritance using interfaces
    public void displayA() {
        System.out.println("Method from A");
    }

    public void displayC() {
        System.out.println("Method from C");
    }
}

public class Main {
    public static void main(String[] args) {
        D obj = new D();

        obj.displayA();
        obj.displayC();
    }
}
```

Method from A
Method from C

